

React 18 并发渲染机制深度解析

React 18 并发渲染机制深度解析

文档类型：技术深度分析 适用版本：React 18.0+ 作者：前端架构组

最后更新：2026-07-10**文档类型**：技术深度分析

适用版本：React 18.0+

作者：前端架构组

最后更新：2026-07-10

文档类型：技术深度分析 适用版本：React 18.0+ 作者：前端架构组

最后更新：2026-07-10**文档类型**：技术深度分析

适用版本：React 18.0+

作者：前端架构组

最后更新：2026-07-10

1. 概述

React 18 引入了并发渲染（Concurrent Rendering）机制，这是自 React 16 引入 Fiber 架构以来最重大的内部革新。并发模式允许 React 在渲染过程中中断、暂停和恢复工作，从而提供更流畅的用户体验。

1.1 核心概念

概念	说明	引入版本
Concurrent Mode	使渲染可中断的新架构模式	React 18
Fiber	虚拟调用栈帧，支持增量渲染	React 16
Suspense	声明式等待异步数据加载	React 16.6+
Time Slicing	将长任务切成多个小时间片	React 18
Automatic Batching	自动批量更新状态	React 18
Transitions	区分紧急和非紧急更新	React 18

1.2 设计目标

- 流畅交互：确保用户输入永远不会被长时间渲染任务阻塞
- 渐进式升级：现有应用可以平滑迁移，无需大规模重写
- 自适应调度：根据设备性能和网络状况动态调整渲染优先级

2. 架构演进：从 Stack Reconciler 到 Fiber2. 架构演进：从 Stack Reconciler 到 Fiber

2.1 Stack Reconciler 的局限性2.1 Stack Reconciler 的局限性

```
[=====  =====]  
↑                               ↑
```

当组件树较大时，递归调用栈会长时间占用主线程，导致页面卡顿。当组件树较大时，递归调用栈会长时间占用主线程，导致页面卡顿。

2.2 Fiber 架构2.2 Fiber 架构

React 16 重构了调和引擎，引入 Fiber 节点：React 16 重构了调和引擎，引入 Fiber 节点：

```
// Fiber  
interface FiberNode {  
  //  
  tag: WorkTag;           // FunctionComponent | ClassComponent | HostRoot ...  
  type: any;              //  
  key: null | string;  
  
  //  
  return: Fiber | null;    //  
  child: Fiber | null;     //  
  sibling: Fiber | null;    //  
  
  //  
  pendingProps: any;  
  memoizedProps: any;  
  memoizedState: any;  
  updateQueue: UpdateQueue | null;  
  
  //  
  effectTag: Flags;        // DOM  
  nextEffect: Fiber | null;  
  
  //  
  lanes: Lanes;           // Fiber  
  childLanes: Lanes;  
}
```

Fiber 将虚拟 DOM 树转化为链表结构，使 React 能够：

- 在任何 Fiber 节点暂停工作
- 在任何 Fiber 节点暂停工作
- 根据优先级丢弃已完成的工作
- 根据优先级丢弃已完成的工作
- 为新到来的高优先级更新让路
- 为新到来的高优先级更新让路

3. 并发特性详解3. 并发特性详解

3.1 createRoot API3.1 `createRoot` API

React 18 引入了新的根节点 API：React 18 引入了新的根节点 API：

```
// React 17
import ReactDOM from 'react-dom';
ReactDOM.render(<App />, document.getElementById('root'));

// React 18 -
import { createRoot } from 'react-dom/client';
const root = createRoot(document.getElementById('root'));
root.render(<App />);
```

3.2 startTransition：区分更新优先级3.2 `startTransition`：区分更新优先级

```
import { useState, startTransition } from 'react';

function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  function handleInput(e) {
    //
    setQuery(e.target.value);

    //
    startTransition(() => {
      const filtered = searchData(e.target.value);
      setResults(filtered);
    });
  }

  return (
    <div>
      <input value={query} onChange={handleInput} />
      <ResultList data={results} />
    </div>
  );
}
```

3.3 useDeferredValue3.3 `useDeferredValue`

用于延迟接收某个值的更新，避免阻塞更紧急的渲染：用于延迟接收某个值的更新，避免阻塞更紧急的渲染：

```
import { useState, useDeferredValue } from 'react';

function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  // deferredQuery query
  const results = useMemo(
    () => performExpensiveSearch(deferredQuery),
    [deferredQuery]
  );

  return <ResultList loading={query !== deferredQuery} data={results} />;
}
```

3.4 Suspense 增强3.4 Suspense 增强

```
<Layout>
  <Suspense fallback={<Spinner />}>
    <Comments />
  </Suspense>
  <Suspense fallback={<SidebarSkeleton />}>
    <Sidebar />
  </Suspense>
</Layout>
```

4. 调度优先级系统4. 调度优先级系统

4.1 Lane 模型4.1 Lane 模型

React 18 使用 Lane 模型替代了 React 17 的 `expirationTime`。

优先级等级	数值	用途
SyncLane	0b1	同步渲染，不可中断
InputContinuousLane	0b100b100	连续输入事件（拖拽、滚动）
DefaultLane	0b100000b10000	默认优先级
TransitionLane	0b10000000b1000000	startTransition 产生`startTransition`
IdleLane	0b1000000000b100000000	空闲时执行

4.2 调度流程4.2 调度流程



5. 自动批处理 (Automatic Batching) 5. 自动批处理 (Automatic Batching)

```
// React 17 -
setTimeout(() => {
  setCount(c => c + 1); //
  setFlag(f => !f);    // 2
}, 1000);

// React 18 -
setTimeout(() => {
  setCount(c => c + 1);
  setFlag(f => !f);
  // React 18 1
}, 1000);
```

5.1 退出批处理5.1 退出批处理

```
import { flushSync } from 'react-dom';

flushSync(() => {
  setCount(c => c + 1);
});
// DOM
flushSync(() => {
  setFlag(f => !f);
});
// DOM
```

6. 性能基准测试6. 性能基准测试

6.1 测试环境6.1 测试环境

指标指标	配置配置
CPUCPU	Apple M2 ProApple M2 Pro
内存内存	16 GB16 GB
浏览器浏览器	Chrome 120Chrome 120
测试场景测试场景	10000 列表项渲染10000 列表项渲染

6.2 测试结果6.2 测试结果

指标指标	React 17React 17	React 18 (Concurrent)React 提升提升
首次渲染 (ms)首次渲染 (ms)	245245	2382382.9%2.9%
输入响应延迟 (ms)输入响应延迟	187187	424277.5%77.5%
FCP (ms)FCP (ms)	320320	2952957.8%7.8%
TTI (ms)TTI (ms)	860860	52052039.5%39.5%
长任务数量长任务数量	1414	3378.6%78.6%

7. 迁移指南7. 迁移指南

7.1 兼容性检查清单7.1 兼容性检查清单

- [] 升级 react 和 react-dom 到最新 18.x 版本[] 升级 `react` 和 `react-dom` 到最新 18.x 版本
- [] 将 ReactDOM.render 替换为 createRoot[] 将 `ReactDOM.render` 替换为 `createRoot`
- [] 移除 React.StrictMode 的旧版行为（可选）[] 移除 `React.StrictMode` 的旧版行为（可选）
- [] 检查第三方库兼容性[] 检查第三方库兼容性
- [] 将需要延迟的更新包裹在 startTransition 中[] 将需要延迟的更新包裹在 `startTransition` 中

7.2 常见陷阱7.2 常见陷阱

注意：useEffect 在 Strict Mode 下会执行两次（仅开发环境），这是 React 18 的设计行为，用于检测副作用是否正确清理。**** 注意****：`useEffect` 在 Strict Mode 下会执行两次（仅开发环境），这是 React 18 的设计行为，用于检测副作用是否正确清理。

注意：useEffect 在 Strict Mode 下会执行两次（仅开发环境），这是 React 18 的设计行为，用于检测副作用是否正确清理。**** 注意****：`useEffect` 在 Strict Mode 下会执行两次（仅开发环境），这是 React 18 的设计行为，用于检测副作用是否正确清理。

8. 总结8. 总结

React 18 的并发特性标志着 React 从同步确定性渲染向自适应优先级调度的范式转变。核心改进包括：

1. 可中断渲染：主线程永远不会被长渲染任务锁死**可中断渲染**：主线程永远不会被长渲染任务锁死
2. 优先级区分：紧急交互优于非紧急页面更新**优先级区分**：紧急交互优于非紧急页面更新
3. 自动批处理：无需手动优化即可获得更好的性能**自动批处理**：无需手动优化即可获得更好的性能
4. Suspense 增强：更优雅的异步数据处理**Suspense 增强**：更优雅的异步数据处理

推荐阅读：React 官方文档 - Concurrent Features **推荐阅读**：[React 官方文档 - Concurrent Features](https://react.dev/blog/2022/03/29/react-v18)

推荐阅读：React 官方文档 - Concurrent Features **推荐阅读**：[React 官方文档 - Concurrent Features](https://react.dev/blog/2022/03/29/react-v18)

本文档仅供内部技术参考，未经授权不得外传。